

Prueba tecnica para BD – Brandon Jimenez Rodriguez

Para ejecutar el proyecto es indispensable clonar el repositorio desde github:

git clone <https://github.com/brandhi/idtest.git>

y consultar el archivo **README.md**

1. Dado el archivo .csv que contiene los datos de 4 entidades (CDMX, Morelos, Edo. México y Querétaro) Diseñar una DB que contenga toda la información de manera que pueda ser accesible, organizada e indexada.

a) Diseñar el MER de la DB para almacenar los datos.

El archivo DDL_IDTEST.sql contiene las instrucciones DDL para la creación del esquema con la logica definida.

b) Se puede programar un script que haga la carga masiva al schema (puede usarse Python, PHP o el lenguaje de su gusto o su dominio)

El scrip generado con el nombre etL_down_up.py contiene la logia en python que:

1. Genera 3 archivos csv apartir del csv de insumo BD_PRUEBA_.csv
2. Hace la carga masiva de datos a la BD tomando como insumo los 3 csv nuevos del punto anterior.
3. Finalmente se generan dos archivos el .dump y .sql , cualquiera de estos 2 se puede usar para montar la BD.

c) Diseñar los SP que ayuden a aprovisionar los resultados que solvente los siguientes reportes:

a. Listado del top 10 de colonias que tengan mayor población femenina de 15 a 49 años.

1. Hallazgo Técnico y Diagnóstico de Datos

Nota sobre los datos censales (Diagnóstico):

Al analizar la fuente original BD_PRUEBA_.csv, se identificó que las métricas demográficas (como p_15a49_f y p_total) contienen valores consolidados a nivel Entidad Federativa repetidos en cada asentamiento geográfico. Por ejemplo, registros individuales de colonias en la CDMX muestran la cifra del estado completo (8,851,080 en p_total y 2,495,688 en p_15a49_f). Aplicar una agregación directa SUM() sobre esta estructura genera un producto cartesiano que infla los totales a decenas de millones.

2. Propuesta(Lógica del Stored Procedure)

Propuesta de Implementación :

Asumiendo que la columna fbi_p_15a49_f representa el total ya calculado de población femenina de 15 a 49 años por asentamiento/colonia, la lógica correcta consiste en obtener la

métrica escalar por colonia mediante MAX(fbi_p_15a49_f) (para evitar duplicaciones en caso de joins con múltiples geografías) y ordenar de manera descendente para extraer el Top N configurado en el parámetro p_limit.

Archivo : dql/sp_a.sql

b. Datos absolutos y porcentuales por entidad (estado) de población con alguna limitación/discapacidad específica.

Hallazgo Técnico y Criterio de Agregación Metodológico

Se asume que el suministro de datos cuenta con las métricas demográficas (p_total, pcon_lim, pclim_*) precalculadas y consolidadas a nivel Entidad Federativa (Estado).

Por esta razón, se utiliza la función de agregación MAX() agrupada por estado, evitando la duplicidad o sobreconteo (fan-out) que ocurriría al aplicar SUM() sobre las distintas geometrías/colonias asociadas a una misma entidad.

Archivo : dql/sp_b.sql

c. Datos absolutos en total por entidad de los datos del diccionario de las columnas 10,11,12,22 y 26

Hallazgo Técnico y Criterio de Agregación Metodológico

Se aplica MAX() asumiendo valores consolidados a nivel Entidad Federativa. En caso de consultar la información con granularidad a nivel localidad, colonia o AGEB, correspondería utilizar la agregación SUM().

Archivo : dql/sp_c.sql

Nota: al revisar los datos, puedo notar que los valores vienen consolidados por entidad federativa, no vienen consolidados por municipio. Construí los Sps tomando en cuenta que vienen consolidados por entidad federativa excepto el inciso A.

Revisando datos similares en el sitio oficial del INEGI:

https://www.inegi.org.mx/programas/ccpv/2020/#Datos_abiertos

veo que los datos si deberían venir consolidados por municipio y no por entidad federativa.

d. Insumos necesarios:

Para integrar la capa espacial a la base de datos se requieren los siguientes componentes:

A. Archivos vectoriales (Shapefiles/ GeoJSON/ GeoPackage):

Geometría de entidades federativas (límite estatal): Polígonos de los 32 estados.

Geometría de localidades/colonias/AGEB: Polígonos o Puntos de las localidades censales con su llave geográfica única (CVEGEO).

B. Sistema de referencia espacial (SRID/EPSG)

Para México, el estándar oficial es **EPSG:6372** (WGS 84 / ITRF2008 / UTM) o **EPSG:4326** (WGS 84 – Latitud/Longitud).

C. Extensión Espacial Activada

en este caso ya la tenemos activada en la bd postgresql .

Proceso para la integración:

1. Confirmar que la extensión esta activada.

2. Cargar los archivos de datos geograficos a la BD usando herramientas como ogr2ogr

Ejemplo usando ogr2ogr desde la terminal

```
ogr2ogr -f "PostgreSQL" PG:"host=localhost dbname=mi_bd user=postgres password=secret"
"conjunto_de_datos/00ent.shp" -nln idtest.cid_state_geom -a_srs "EPSG:6372" -t_srs
"EPSG:4326"
```

3. Estandarización e indexación de la columna geometrica.

Convertir la columna de geometría a tipo GEOMETRY(Geometry, 4326) y agregar un indice sobre la misma columna tipo GEOMETRY, en el caso de nuestro DDL ya se incluyo esta columna CGO_GEOM en la tabla IDTEST.CID_GEOGRAPHY , al igual que el indice espacial ya esta creado.

-- Ejemplo:

-- Agregar columna de geometría si se trabaja sobre una tabla existente

```
ALTER TABLE IDTEST.CID_GEOGRAPHY ADD COLUMN IF NOT EXISTS CGO_GEOM
GEOMETRY(Geometry, 4326);
```

-- Crear índice espacial GiST (imprescindible para desempeño)

```
CREATE INDEX IX2_CID_GEOGRAPHY_GEOM ON IDTEST.CID_GEOGRAPHY USING
GIST(CGO_GEOM);
```

4. adicionalmente se pude hacer una validación y reparacion de topología.

Verificar que las geometrías cargadas sean válidas (sin polígonos auto-intersecados o huecos en las fronteras).

-- Corregir geometrías no válidas automáticamente

```
UPDATE IDTEST.CID_GEOGRAPHY
```

```
SET geom = ST_MakeValid(geom)
```

```
WHERE NOT ST_IsValid(geom);
```

Nota: de esta forma ya podemos consultar los datos espaciales y crear algunas vistas útiles.